

Case Study: Online Shopping Cart Management

Scenario

An e-commerce website stores the products added to a customer's shopping cart. Each product contains information such as its ID, name, category, price, quantity, and availability.

Your task is to use **JavaScript array methods** to analyze and manipulate the shopping cart.

Shopping Cart Data

```
const cart = [  
  {  
    id: 101,  
    name: "Laptop",  
    category: "Electronics",  
    price: 55000,  
    quantity: 1,  
    inStock: true  
  },  
  {  
    id: 102,  
    name: "Wireless Mouse",  
    category: "Electronics",  
    price: 1200,  
    quantity: 2,  
    inStock: true  
  },  
  {  
    id: 103,  
    name: "Office Chair",  
    category: "Furniture",  
    price: 8500,  
    quantity: 1,  
    inStock: false  
  },  
  {  
    id: 104,  
    name: "Notebook",  
    category: "Stationery",  
    price: 120,  
    quantity: 5,  
  }  
]
```

```
        inStock: true
    },
    {
        id: 105,
        name: "Water Bottle",
        category: "Accessories",
        price: 450,
        quantity: 3,
        inStock: true
    }
];
```

Activities

Activity 1 – Display Product Names (**forEach**)

Display the name and price of every product in the cart.

Example Output

```
Laptop - ₹55000
Wireless Mouse - ₹1200
Office Chair - ₹8500
Notebook - ₹120
Water Bottle - ₹450
```

Activity 2 – Calculate Total Price (**map**)

Create a new array that contains:

```
Product Name
Total Price (price × quantity)
```

Expected Structure

```
[
    { name: "Laptop", total: 55000 },
    { name: "Wireless Mouse", total: 2400 },
    ...
]
```

Activity 3 – Find Available Products (**filter**)

Display only the products that are currently **in stock**.

Activity 4 – Find Expensive Products (**filter**)

Display products whose price is greater than ₹5000.

Activity 5 – Search for a Product (**find**)

Find the product with:

ID = 103

Display the complete product details.

Activity 6 – Check Product Availability (**some**)

Check whether **any product** is out of stock.

Expected Output

`Yes, some products are out of stock.`

Activity 7 – Verify Stock Status (**every**)

Check whether **all products** are available.

Expected Output

`All products available: false`

Activity 8 – Calculate Grand Total (**reduce**)

Calculate the total bill amount of the cart.

Formula:

`price × quantity`

Expected Output

Grand Total = ₹68990

Activity 9 – Sort Products by Price (**sort**)

Sort the products in **ascending order of price**.

Display:

Notebook
Water Bottle
Wireless Mouse
Office Chair
Laptop

Activity 10 – Display Only Product Names (**map**)

Create a new array containing only the product names.

Expected Output

```
[  
  "Laptop",  
  "Wireless Mouse",  
  "Office Chair",  
  "Notebook",  
  "Water Bottle"  
]
```

Activity 11 – Find Product Position (**findIndex**)

Find the index of the product named **Office Chair**.

Activity 12 – Remove Out-of-Stock Products (**filter**)

Create a new cart containing only available products.

Activity 13 – Add GST (**map**)

Create a new array where every product contains an additional property:

gst = 18% of price

Activity 14 – Top 3 Cheapest Products

Use:

```
sort()  
slice()
```

Display the three cheapest products.

Activity 15 – Generate Bill Summary (**reduce**)

Generate an object with the following details:

```
{  
  totalProducts: ?,  
  totalQuantity: ?,  
  totalAmount: ?  
}
```

JavaScript Array Methods Covered

Method	Purpose
<code>forEach()</code>	Iterate through each product
<code>map()</code>	Transform data into a new array
<code>filter()</code>	Select products based on a condition
<code>find()</code>	Find the first matching product
<code>findIndex()</code>	Find the position of a product
<code>some()</code>	Check if at least one item satisfies a condition
<code>every()</code>	Check if all items satisfy a condition
<code>reduce()</code>	Calculate totals or aggregate values
<code>sort()</code>	Sort products by price
<code>slice()</code>	Extract a portion of the array